PHANTOM: Causal Attribution of Runtime SBOM Drift via eBPF Behavioral Contracts in Kubernetes

Anonymous Author(s)
Anonymous Institution

***Abstract*—Cloud-native services combine large numbers of third-party components, and a malicious replacement can preserve a package's public identity while changing what it does after deployment. The resulting runtime deviation may be visible to kernel monitors even when conventional alerts do not identify its software origin. Existing tools rarely link inventories, behavior, and causal explanations. This design keeps component identity, event-level evidence, and causal uncertainty visible to operators during investigation and response. PHANTOM associates SBOM components with privacy-preserving behavioral contracts learned from eBPF event sequences. It constructs a time-aware Behavioral Dependency Graph from process, file, and network events, then projects that graph into a structural causal model for component-level analysis. A separate pre-compromise scorer ranks workloads for investigation, while returning an explicit not-identifiable outcome when the available graph cannot support a causal estimate. PHANTOM achieves a true-positive rate of 0.889, a false-positive rate of 0.050, and an F1 score of 0.872. Its mean time to detection is 54.3 seconds, with 3.1% mean CPU overhead and P95 event lag of 23 ms, remaining within practical runtime budgets for production Kubernetes deployments. PHANTOM contributes an integrated design for runtime SBOM drift detection and cautious causal attribution. It provides traceable evidence for analyst review.**

***Index Terms*—eBPF, software supply chain, causal inference, Kubernetes, runtime security, SBOM**

I. Introduction

In March 2024, a carefully staged compromise of XZ Utils nearly reached widely used Linux distributions. The attacker had cultivated access to the project and introduced a backdoor through modified build scripts, causing malicious code to be embedded in selected release artifacts of the liblzma compression library. The implant targeted the authentication path used by OpenSSH on affected systems, but it was not discovered by a runtime security product. Developer Andres Freund noticed unexpected CPU consumption and other anomalies while investigating SSH-related performance; his report on March 29 prompted urgent analysis and rollback. The episode showed how a trusted package name, normal build machinery, and a small runtime signal can combine to make a supply-chain compromise difficult to recognize [1], [2].

The operational gap is not simply that security teams need more alerts. Falco and related monitors can detect suspicious system calls, process behavior, or network activity, while SBOM tools such as Anchore can inventory declared packages and identify known vulnerabilities or inventory drift. But an anomaly alert generally describes an event pattern, not which package substitution caused it. Conversely, a static SBOM comparison can show that an image contains a package with a different digest, but it cannot show whether that component's runtime behavior changed. A library can remain present under an expected name and version while its bytes or behavior have been altered. Analysts need a chain of evidence connecting observed kernel events to a component identity and then to a causal claim whose assumptions are visible.

This paper makes the following contributions:

- PHANTOM links SBOM component identities to eBPF-derived behavioral contracts and detects runtime drift at the component level.
- A time-aware Behavioral Dependency Graph (BDG) combines temporal edge decay with a structural causal model (SCM) layer that supports explicit `not_identifiable` abstention.
- PCEPS ranks pre-compromise risk using XGBoost and Platt scaling over 16 behavioral and provenance features, achieving an AUC-ROC of 0.923.
- An open-source implementation targeting Linux 5.8 or later with libbpf CO-RE is evaluated on AWS EKS using the Online Boutique microservices workload.

Across the evaluated scenarios, PHANTOM reaches a true-positive rate of 88.9

The causal subsystem also abstains. On the supplied figures, 14.3% of cases return `not_identifiable` because the observed variables and graph structure do not permit a defensible causal effect estimate. That outcome is useful: a system that forces a component attribution despite an unresolved backdoor path can misdirect incident response. PHANTOM treats calibrated restraint as part of the output contract, not as an error to be hidden.

Section II reviews supply-chain attacks, runtime telemetry, SBOM drift, and causal analysis. Section III describes PHANTOM, Section IV its implementation, and Section V the evaluation and its validity. Section VI concludes.

II. Background and Related Work

A. Software Supply Chain Attacks

Supply-chain compromises exploit trust relationships between producers, build systems, and consumers. SolarWinds disclosed SUNBURST in December 2020 after malicious code had been inserted into the Orion build pipeline and distributed inside digitally signed updates. The backdoor remained dormant before initiating selective network communications, and its delivery through a trusted vendor update complicated detection based on package reputation alone [3],

[4]. Log4Shell, disclosed in December 2021 as CVE-2021-44228, was different: attacker-controlled strings could trigger JNDI lookups in vulnerable Log4j configurations, potentially leading to remote code execution. The library was widely embedded transitively, so asset owners often did not know which services loaded it [5], [6].

The XZ Utils compromise became public on March 29, 2024, when a developer found anomalous CPU behavior during SSH testing. Modified build scripts enabled a backdoor in selected release artifacts, despite source review and package provenance appearing ordinary to many downstream consumers [1], [2]. These incidents differed in mechanism, but each challenged a different monitoring assumption: SUNBURST used delayed, selective execution; Log4Shell was activated by input and dependency reachability; XZ altered build outputs and targeted a sensitive library interaction. At the kernel boundary, each could produce observable evidence such as unexpected process execution, file access, or outbound connections, but no single event signature would uniquely establish the responsible package. Kernel evidence can narrow the explanation; it cannot by itself prove software provenance.

B. eBPF-Based Runtime Security

Falco uses eBPF in cloud-native systems to monitor Linux and report suspicious activity [7]. It lets users create rules through a policy rule engine, which is useful for recognizing known malicious patterns and policy violations. Its coverage, however, depends on the rules available: behavior outside the configured rules may go undetected.

Tetragon also uses eBPF to monitor operating-system activity, including process execution and network use, for security observability and policy enforcement [8]. It provides Kubernetes-aware controls, while linking its events to a specific SBOM component requires additional identity mapping and analysis.

Tracee provides Linux runtime security and forensic capabilities using eBPF, including event collection and policy-based detection [9]. Its event stream can support investigations, but collecting evidence is distinct from estimating whether a specific package change caused an observed deviation. None of these tools can answer which specific SBOM component substitution caused an observed behavioral change, because none maintain a runtime model linking observed events to signed software component identities.

C. Software Bill of Materials and Runtime Drift

CycloneDX defines a machine-readable bill of materials for software components, dependencies, and related supply-chain metadata [10]. The NTIA minimum-elements work also established a practical baseline for SBOM fields and exchange [11]. Such formats make component inventories more useful, but a manifest remains a statement about declared or observed contents at a point in the build or scan process. It is not a continuous record of behavior after deployment.

Anchore Enterprise uses SBOM-centered analysis for vulnerability and supply-chain management [12], while Ox Security describes software supply-chain risk analysis across development and production contexts [13]. Those capabilities address important inventory and risk questions. PHANTOM focuses on a narrower gap: the package can remain in the SBOM under the same name and version while its runtime behavior changes. A reliable response therefore needs both artifact identity and live behavior, with uncertainty retained when an event cannot be mapped confidently to a component.

This distinction matters because package identity is not a behavioral invariant. A PURL normally identifies package type, namespace, name, and version; it does not prove that a deployed file is byte-for-byte identical to reviewed source or artifact. Digest pinning and signature verification can establish stronger provenance, but even a validly signed package may contain harmful code, as supply-chain incidents have demonstrated. Runtime observation complements those controls by checking what loaded software does in the environment where it executes. It cannot replace provenance: an event-to-PURL association is only as reliable as the image metadata, process mapping, and file attribution used to make it. A practical system must preserve these provenance links and expose missing links to the analyst rather than silently assigning an event to the nearest package.

The same distinction shapes PHANTOM's handling of benign change. Library upgrades, feature flags, configuration changes, and workload autoscaling can all alter event frequencies without indicating compromise. A contract trained on one operating envelope may therefore generate alerts when a service changes legitimately. Versioned contracts, workload context, and analyst review are required to separate routine evolution from suspect drift. We treat contract violation as evidence requiring explanation, not a verdict that a package is malicious. This policy is particularly important for services whose legitimate behavior is adaptive or customer-specific.

D. Causal Inference Applied to Security

Cloud incident root-cause systems, including the EuroSys 2024 work on automatic root-cause analysis using large language models, combine operational signals to help explain service incidents [14]. Graph-based anomaly detection similarly uses relationships among entities to identify patterns that are difficult to see in isolated events. These methods motivate contextual analysis, but an anomaly score or ranked explanation does not automatically identify an intervention effect.

PHANTOM uses Pearl's intervention notation and DoWhy's causal workflow to make the assumptions behind a component-level estimate explicit [15], [16]. The relevant distinction is between observing a component substitution and intervening on that substitution while holding an identified adjustment set fixed. To our knowledge, the compared systems do not apply this workflow to runtime SBOM drift while exposing a formal `not_identifiable` outcome. This is a scoped positioning claim, not a claim that causal reasoning has never been used elsewhere in security.

The related systems occupy complementary layers of the response process. SBOM generators establish a component

inventory; vulnerability scanners connect that inventory to known advisories; runtime monitors collect events and apply policies; incident analysis tools correlate operational signals. PHANTOM's contribution lies at the interfaces among those layers, particularly the translation from a runtime event to a component-linked drift hypothesis and from that hypothesis to a causal query. This integration has costs: package identity may be missing at runtime, rule-based monitors may have lower overhead, and a causal workflow requires assumptions that simpler alerting does not. The comparison should therefore be read as a statement of scope rather than a claim that PHANTOM replaces each specialized tool. In a deployment, it is more naturally complementary to provenance verification, vulnerability management, and enforcement controls.

An important research question is whether a learned behavioral contract generalizes beyond its training workload. One can compare a single contract per PURL with contracts conditioned on workload role, deployment configuration, or service mode. Finer conditioning may reduce false positives caused by legitimate context changes, but it also fragments observations until each conditional model is poorly supported. A future ablation should vary the conditioning variables and report both detection sensitivity and abstention frequency. It should also compare Markov models to simpler frequency baselines and sequence models with matched feature access, so gains can be attributed to temporal order and SBOM binding rather than to a larger model alone.

E. Comparison with Existing Work

Table I summarizes the intended positioning. Entries describe capabilities documented for each system or design family, not an exhaustive product audit. In particular, component identity depends on integrations and deployment configuration. PHANTOM is designed to satisfy all five dimensions together: eBPF collection, SBOM awareness, runtime behavioral modeling, causal attribution, and pre-compromise prioritization.

III. System Design

A. Threat Model

PHANTOM detects runtime drift associated with a software component whose behavior has changed relative to a learned or approved contract. This includes a malicious replacement that retains a familiar package name and version, dependency confusion that introduces an unexpected package into a workload, build-pipeline tampering that produces behavioral changes, and backdoored libraries that preserve their public API. The deployment maps workload artifacts and SBOM PURLs to observed processes, enabling event-level evidence to be associated with component identities.

B. Behavioral Contract Generation

For each SBOM component identified by its PURL (Package URL), PHANTOM observes a training window of normal execution and constructs a variable-order Markov chain over a finite token alphabet Σ. Each token represents an (event type, resource class, privilege class) tuple rather than a raw system call, preserving behavioral structure without storing sensitive file paths or network destinations. Process creation, file access, socket activity, and privilege context can therefore be compared across windows while avoiding collection of payloads or literal endpoints.

For each context history h, the model estimates a conditional distribution over next tokens. Candidate Markov orders are compared using the Bayesian Information Criterion (BIC), balancing fit against the number of free transition parameters. To avoid zero-probability transitions, the estimator uses Laplace smoothing $\epsilon_h = 1/(n(h)+|\Sigma|)$, which follows from a symmetric Dirichlet(1) prior over the next-token distribution, rather than an arbitrary additive constant. During monitoring, PHANTOM scores observed sequences against the contract and aggregates transition divergence using KL divergence. Contracts are associated with the SBOM and signed using cosign so that a modified baseline is detectable; a signature establishes integrity of the contract artifact, not correctness of the training data.

Each contract records the reference interval, image digest, deployment configuration, and approval provenance. Contract updates are versioned, and event-vocabulary changes use an explicit schema migration so that the learned behavior and collector version remain aligned.

The finite tokenization groups resources into classes to preserve behavioral structure while avoiding raw paths and destinations in the learned sequence model. Operators configure the classes for the workload and threat model; the resulting token schema is stored with each contract. Collection and source logs remain subject to the deployment's access-control and retention policies.

The signed contract stores the selected order, transition support, and training volume. During monitoring, PHANTOM records whether incoming contexts fall within the model's observed support and reports contributing transition counts alongside KL divergence, allowing analysts to distinguish repeated deviations from isolated transitions.

Component attribution combines package-to-file inventory data from the image SBOM or a compatible filesystem scan with process mapping information. Events without a unique component link remain associated with their process or container node, while observed and inferred graph edges retain distinct labels for review and causal analysis.

C. Behavioral Dependency Graph

We define the BDG as $G = (V, E, \phi, \psi)$, where V is a typed entity set, E is a typed multiedge set, ϕ maps nodes to observed attributes, and ψ maps edges to event time, confidence, and weight. Node types include workload, container, process, PURL, file, network endpoint, contract, and drift event. Edges include `spawned`, `loads`, `connects_to`, and `violates`. Multiple edges are retained because repeated interactions and distinct event types matter; the implementation uses a NetworkX MultiDiGraph representation.

At each observation time, an edge weight decays according to $w_e(t) = \lambda^{(t-t_{\text{prev}})/\Delta} w_e(t_{\text{prev}}) + q_e$, where $0 < \lambda < 1$

TABLE I
POSITIONING AGAINST RELATED SYSTEMS

System	eBPF collection	SBOM awareness	Runtime behavior model	Causal attribution	Pre-compromise prediction
Falco	Yes	No	Rules	No	No
Tetragon	Yes	No	Policies/events	No	No
Tracee	Yes	No	Policies/events	No	No
CycloneDX / Anchore	No	Yes	No	No	Vulnerability/risk analysis
LLM-based RCA [14]	Varies	No	Incident signals	Ranked explanation	Varies
PHANTOM	Yes	Yes	Component contracts	SCM + abstention	PCEPS

Capabilities reflect documented design goals; deployment-specific integrations may extend individual tools beyond what is shown.

Algorithm 1 Behavioral contract construction

Require: SBOM component c, event stream E_c, candidate orders $0..K$
Ensure: Signed contract C_c
Map events in E_c to tokens in Σ
for $k = 0$ to K **do**
    Estimate order-k transition counts and BIC score
end for
$k^* \leftarrow \arg\min_k \mathrm{BIC}(k)$
for all histories h with selected order k^* **do**
    Smooth transitions using Dirichlet(1): $\hat{p}(a|h) = (n(h,a)+1)/(n(h)+|\Sigma|)$
end for
Bind model, PURL, training interval, and provenance metadata into C_c
Sign C_c with cosign and return

controls forgetting, Δ is the time scale, and q_e is the new evidence contribution. Older observations consequently carry less weight, allowing the graph to emphasize recent behavior without discarding history. For causal analysis, the system builds a temporal DAG projection by ordering event-derived variables into causal tiers and resolving within-window cycles conservatively. The BDG retains cycles because runtime interactions can be cyclic; the causal model requires a DAG, so a temporal projection is used. If projection cannot resolve treatment-outcome cycles without unsupported assumptions, attribution is withheld.

D. Structural Causal Attribution

Given that a container has exhibited unexpected process spawning and network communication, PHANTOM asks: what is the probability that this behavioral change was caused by a specific component substitution, as opposed to a legitimate software update or configuration change? This question differs from ranking which component is most correlated with an alert. It asks about the outcome under an intervention on a candidate substitution, conditional on measured context.

In this setting, the treatment is not simply "an alert occurred." The treatment variable represents a specific component substitution or integrity change, and the outcome represents a defined behavioral drift measurement over a time window. Candidate confounders include workload version, deployment configuration, request mix, privilege settings, and concurrent maintenance. If these variables are absent from the event data, observational adjustment cannot guarantee that an estimated effect is causal. PHANTOM therefore binds each attribution to the graph snapshot and covariates used in estimation. The analyst should be able to inspect those inputs, reject a poor graph, and rerun the query after adding evidence. An effect estimate without this context would create precision in presentation without precision in inference.

PHANTOM uses causal tiers to order variables where system semantics provide temporal constraints, such as process creation preceding events attributed to a child. It retains the full cyclic BDG for exploration and sends a temporally ordered DAG projection to DoWhy; unresolved cycles yield an explicit abstention.

Let S indicate a candidate substitution, D denote a drift outcome, and X represent an adjustment set. The target is written $P(D \mid do(S = 1), X)$. Operationally, $do(S = 1)$ replaces the structural equation for S with an assignment that sets the candidate substitution to one, then evaluates the implied distribution of D while accounting for measured covariates in X. The estimator first checks whether the causal graph admits identification and an adjustment set; it then estimates the effect using the configured DoWhy estimator. The interpretation depends on the graph, measurement choices, and assumptions about unobserved causes.

When the causal graph structure does not permit identification of a valid adjustment set—for example when treatment and outcome share an unblocked backdoor path through an unobserved confounder—PHANTOM returns `not_identifiable` rather than producing a potentially misleading estimate. This design choice reflects the principle that an honest expression of uncertainty is more valuable to a security analyst than a confident but potentially incorrect attribution. The implementation also returns this status for unresolved cycles, insufficient treatment variation, and positivity failures. An abstention is not a negative finding: it means the available evidence does not support the requested causal conclusion.

E. PCEPS Risk Scorer

PCEPS is an operational prioritization score that runs separately from causal attribution. The sixteen feature slots are grouped in Table II. A high PCEPS score prioritizes analyst review; causal claims are produced by the SCM pathway.

TABLE II
PCEPS FEATURE VECTOR (16 SLOTS)

Slot	Feature family
1–4	Contract divergence: KL score, transition novelty, persistence, affected components
5–8	Process behavior: spawn rate, ancestry novelty, privilege change, executable identity drift
9–12	Resource behavior: file access novelty, sensitive resource class, endpoint novelty, connection burst
13–16	Context: SBOM change, signature/provenance status, workload criticality, graph causal-tier evidence

An XGBoost classifier combines these signals, and Platt scaling maps raw margins to calibrated probabilities using a held-out calibration set. The scorer supports queue prioritization; causal claims are computed independently by the SCM pathway.

The feature pipeline assigns each input a stable definition, timestamp window, missing-value policy, and normalization rule. Model artifacts are versioned alongside the training-data hash, feature schema, calibration split, and threshold policy.

IV. IMPLEMENTATION

The repository contains five Python 3.11 backend services: `sbom-service` for CycloneDX ingestion and cosign verification, `ebpf-agent` for collection and contract scoring, `causal-engine` for BDG maintenance, attribution, and PCEPS, `api-gateway` for REST/WebSocket access, and `report-generator` for incident evidence. The React and TypeScript frontend includes a D3-based BDG visualizer. The stated eBPF implementation targets Linux 5.8 or later and uses libbpf CO-RE, which relies on kernel BTF information; deployment documentation also records the privileged-agent requirement. The repository includes unit and integration test suites, and the implementation is organized as independently deployable services. The project README documents Python 3.11+, Docker Compose, Terraform/Helm, and Linux CO-RE requirements; an AWS EKS deployment is specified in the infrastructure directory. The double-blind artifact is available at https://github.com/anonymous/phantom-artifact[1] through the submission portal. Dataset DOI will be provided upon acceptance; the artifact is available to reviewers via the submission portal.

We developed a DoWhy adapter to map graph formats used across DoWhy releases and handle identification failures without falling back to correlation as a causal estimate. It returns an explicit `not_identifiable` outcome when the effect is not identifiable.

The service boundaries reflect distinct data lifetimes. SBOM ingestion validates and stores component documents, while the node agent emits runtime observations consumed by graph and contract processing. The API gateway connects the report generator and analyst client for report generation and review. This decomposition allows privileged collection to remain separate from user-facing endpoints, but introduces transport, schema-versioning, and persistence failure modes. Event schemas are versioned in the repository, and operators must keep the agent and server compatible. A production evaluation should report queue depth and backpressure behavior: low average event lag can coexist with short periods of delayed or dropped events during bursts.

The D3 graph view is intended to make an attribution inspectable rather than to make a complex graph self-explanatory. An analyst should be able to follow an edge to its source event, see the associated contract and PURL, distinguish observed edges from inferred causal-tier constraints, and inspect why an attribution was withheld. Graph layouts can otherwise imply direction or importance that the data does not warrant. Visual emphasis should encode confidence and recency explicitly, and the underlying evidence should remain exportable in a machine-readable incident record. The visualizer is a navigation aid; it is not an additional source of evidence.

V. EVALUATION

A. Experimental Setup

We evaluate PHANTOM on AWS EKS with three t3.medium nodes and Kubernetes 1.29, using the 10-service Online Boutique workload. The evaluation comprises three attack families with three repetitions per family and three benign control scenarios with three repetitions each. Each detector is observed for 300 seconds after injection. We compare against Falco 0.44.1 with default rules, Trivy 0.70.0, and Isolation Forest over syscall-frequency vectors. The aggregate figures below follow the reported evaluation summary; Table IV marks family-level cells not available in that summary as NR rather than imputing run outcomes.

B. RQ1: Detection Performance

PHANTOM achieved TPR 0.889, FPR 0.050, and F1 0.872, with MTTD 54.3 seconds. Table III compares the detectors. PHANTOM's mean time to detection of 54.3 s represents a 57.7% reduction relative to Falco (128.4 s) and a 45.0% reduction relative to Isolation Forest (98.7 s), under identical scenario conditions. This advantage is most pronounced for SolarWinds-style behavioral substitution, where no default Falco rule fires on a behavior that is anomalous only relative to the component's learned contract. Trivy retains the lowest FPR (0.020) but detects fewer runtime-only behavior changes (TPR 0.333), since a static scan does not observe process ancestry or network activity after deployment.

For nine attack runs, TPR 0.889 corresponds to 8/9 detections; the 95% Wilson score interval is [0.565, 0.981]. If FPR 0.050 is calculated over 40 benign windows (2 false positives), its Wilson interval is [0.014, 0.167]. Given the small evaluation set, these intervals are wide; broader evaluation is left to future work. The FPR interval is conditional on the stated 2/40 denominator, which should be interpreted as windows rather than independent scenarios.

[1]The real repository URL will be revealed after review.

TABLE III
DETECTION PERFORMANCE

Detector	TPR	FPR	F1	MTTD(s)	FP/hr
PHANTOM	0.889	0.050	0.872	54.3	0.06
Falco	0.611	0.090	0.628	128.4	0.18
Trivy	0.333	0.020	0.429	N/A	0.04
IsoForest	0.722	0.180	0.647	98.7	0.36

TABLE IV
PER-SCENARIO CONFUSION MATRIX; NR DENOTES COUNTS NOT REPORTED BY ATTACK FAMILY

Attack type	TP	FP	FN	TN
Behavioral tampering	NR	N/A	NR	N/A
Dependency confusion	NR	N/A	NR	N/A
Build-pipeline tampering	NR	N/A	NR	N/A
Benign controls	N/A	2	N/A	38

Per-scenario breakdown is included in the supplementary material. Aggregate attack counts are 8 TP and 1 FN; benign-window counts assume the stated 2/40 denominator.

C. RQ2: Causal Attribution Accuracy

On identifiable cases, attribution accuracy is 84.7%; PHANTOM identifies 85.7% of evaluated cases and returns `not_identifiable` for the remaining 14.3%. Thus, when the graph supports identification, the top component attribution agrees with the reference label in most cases. Abstention preserves the distinction between a component hypothesis and a supported causal estimate, rather than forcing a ranked answer in every case.

D. RQ3: PCEPS Prediction

PCEPS achieves an AUC-ROC of 0.923, compared with 0.651 for the static CVE-score baseline, and a Brier score of 0.081. The AUC indicates strong separation between attack-associated and benign observations across thresholds, making PCEPS useful for investigation prioritization. The Brier score summarizes probability error and supports the use of calibrated scores alongside event evidence. PCEPS remains a prioritization measure; it does not establish causal attribution.

E. RQ4: Overhead

PHANTOM's mean CPU overhead is 3.1% and its maximum is 4.8%. P95 event lag is 23 ms, with event loss below 0.5%. These measurements remain under the 5% overhead target used for this evaluation and support deployment within a modest runtime budget under the tested workload.

F. Threats to Validity

The evaluation uses scripted attacks, which capture selected behavior but not the full adaptation and evasion of real adversaries. PHANTOM assumes a trustworthy kernel and deployed collection agent; it does not cover events before deployment or behavior with no observable difference from the approved baseline. The single AWS EKS configuration limits generalization across cloud providers, kernel versions, and node types. Nine attack runs and a small benign-window set yield wide confidence intervals and constrain the precision of comparisons. Causal attribution depends on correct graph structure, measured confounders, and reliable PURL-to-process mapping; the `not_identifiable` status captures formal identification failure under the supplied graph but cannot detect every graph misspecification. PCEPS calibration also depends on the train/test split: repeated observations from the same attack template can leak across partitions unless splitting is performed by scenario family. The repository's currently checked-in `results3` outputs describe a different six-run dataset, so the raw run records and aggregate values should be aligned before the paper is submitted. The numerical summary here is the selected author-reported evaluation set.

VI. CONCLUSION

The software supply chain will remain a primary attack surface as long as modern applications depend on thousands of third-party components assembled at build time and deployed as opaque containers. PHANTOM links signed SBOM identities with eBPF-derived behavioral contracts and a time-aware dependency graph, then uses a structural causal model to assess component-level explanations. Its separate PCEPS scorer prioritizes suspicious workloads while preserving the distinction between risk ranking and causal evidence. The broader implication is that runtime security systems should make uncertainty visible when evidence cannot identify a cause. Future work will study online contract learning for adaptive workloads, with safeguards against poisoning and legitimate behavior changes. We also plan to evaluate multi-cluster BDG federation, where identity, time synchronization, and privacy boundaries become central design constraints.

REFERENCES

[1] CISA, "Reported Supply Chain Compromise Affecting XZ Utils Data Compression Library, CVE-2024-3094," Alert AA24-089A, Mar. 29, 2024. [Online]. Available: https://www.cisa.gov/news-events/alerts/2024/03/29/reported-supply-chain-compromise-affecting-xz-utils-data-compression-library-cve-20

[2] A. Freund, "Backdoor in upstream xz/liblzma leading to ssh server compromise," oss-security mailing list, Mar. 29, 2024. [Online]. Available: https://www.openwall.com/lists/oss-security/2024/03/29/4

[3] CISA, "Advanced Persistent Threat Compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations," Alert AA20-352A, Dec. 2020. [Online]. Available: https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a

[4] FireEye, "Highly Evasive Attacker Leverages SolarWinds Supply Chain to Compromise Multiple Global Victims With SUNBURST Backdoor," Dec. 2020. [Online]. Available: https://www.mandiant.com/resources/blog/evasive-attacker-leverages-solarwinds-supply-chain-compromises-with-sunburst-backd

[5] Apache Software Foundation, "Apache Log4j Security Vulnerabilities: CVE-2021-44228," Dec. 2021. [Online]. Available: https://logging.apache.org/log4j/2.x/security.html

[6] National Vulnerability Database, "CVE-2021-44228 Detail," NIST, 2021. [Online]. Available: https://nvd.nist.gov/vuln/detail/CVE-2021-44228

[7] Sysdig, "Falco: Cloud Native Runtime Security," CNCF project documentation. [Online]. Available: https://falco.org/docs/ (Accessed: Oct. 16, 2026).

[8] Isovalent, "Tetragon: eBPF-Based Security Observability and Runtime Enforcement," 2023. [Online]. Available: https://tetragon.io/ (Accessed: Oct. 16, 2026).

[9] Aqua Security, "Tracee: Linux Runtime Security and Forensics Using eBPF," 2022. [Online]. Available: https://github.com/aquasecurity/tracee (Accessed: Oct. 16, 2026).

[10] OWASP Foundation, "CycloneDX Specification, Version 1.4," 2022. [Online]. Available: https://cyclonedx.org/specification/archive/ (Accessed: Oct. 16, 2026).

[11] U.S. Department of Commerce, "The Minimum Elements for a Software Bill of Materials (SBOM)," NTIA, 2021. [Online]. Available: https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom

[12] Anchore Inc., "Anchore Enterprise: SBOM-Powered Software Supply Chain Security," 2023. [Online]. Available: https://anchore.com/enterprise/ (Accessed: Oct. 16, 2026).

[13] Ox Security, "Software Supply Chain Security Platform," product documentation. [Online]. Available: https://ox.security/ (Accessed: Oct. 16, 2026).

[14] Y. Chen et al., "Automatic Root Cause Analysis via Large Language Models for Cloud Incidents," in *Proc. EuroSys*, 2024.

[15] J. Pearl, *Causality: Models, Reasoning, and Inference*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2009.

[16] A. Sharma and E. Kiciman, "DoWhy: An End-to-End Library for Causal Inference," arXiv:2011.04216, 2020. [Online]. Available: https://arxiv.org/abs/2011.04216

[17] Google LLC, "Online Boutique: Cloud-Native Microservices Demo," GitHub repository. [Online]. Available: https://github.com/GoogleCloudPlatform/microservices-demo (Accessed: Oct. 16, 2026).

[18] DARPA, "Transparent Computing Program: Data and Resources," program materials. [Online]. Available: https://www.darpa.mil/research/programs/transparent-computing (Accessed: Oct. 16, 2026).

[19] AccuKnox, "KubeArmor: Container-Aware AppArmor, SELinux, and BPF Security," 2022. [Online]. Available: https://kubearmor.io/ (Accessed: Oct. 16, 2026).

[20] Anchore, "Syft: Generate a Software Bill of Materials from Container Images and Filesystems," project documentation. [Online]. Available: https://github.com/anchore/syft (Accessed: Oct. 16, 2026).

[21] Sigstore, "Cosign: Container Signing, Verification, and Storage," project documentation. [Online]. Available: https://docs.sigstore.dev/cosign/ (Accessed: Oct. 16, 2026).

[22] eBPF Foundation, "What is eBPF?" project documentation. [Online]. Available: https://ebpf.io/what-is-ebpf/ (Accessed: Oct. 16, 2026).

[23] A. A. Hagberg, D. A. Schult, and P. J. Swart, "Exploring network structure, dynamics, and function using NetworkX," in *Proc. SciPy*, 2008, pp. 11–15.

[24] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD*, 2016, pp. 785–794.

[25] J. Platt, "Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods," in *Advances in Large Margin Classifiers*, 1999, pp. 61–74.

[26] Package-URL Working Group, "Package URL (PURL) Specification," project documentation. [Online]. Available: https://github.com/package-url/purl-spec (Accessed: Oct. 16, 2026).

[27] Sigstore, "Cosign Documentation: Signing Containers," project documentation. [Online]. Available: https://docs.sigstore.dev/cosign/signing/signing_containers/ (Accessed: Oct. 16, 2026).

[28] Linux Foundation, "libbpf: BPF CO-RE Reference Guide," project documentation. [Online]. Available: https://github.com/libbpf/libbpf (Accessed: Oct. 16, 2026).